BVS1 – Betriebssysteme

Winter Semester 2026/2027 · TH Köln · ICCT

Stavros Kousidis

Table of contents

Course Overview **4**
Course Material . . . 4
License . . . 4

1 From Executing Code to Operating Systems **5**
1.1 One program can run without a guest operating system . . . 5
1.1.1 Interpreting the experiment . . . 6
1.2 The ISA connects software and processor . . . 7
1.2.1 Why RV64? . . . 7
1.3 Development host and target are different roles . . . 7
1.4 Virtualization and emulation are different . . . 8
1.4.1 Virtualization: the guest instructions already belong to the host ISA . . . 8
1.4.2 Emulation: the target instruction may not be a host instruction . . . 9
1.4.3 Our BVS1 machine stack . . . 9
1.5 The toolchain connects development host and target . . . 10
1.5.1 Object files are not yet the complete program . . . 11
1.6 When does an operating system become useful? . . . 11
1.7 Applying the working definition . . . 12
1.8 From Hello BVS1 to a minimal operating system . . . 13
1.9 Key ideas . . . 14
Related Exercise . . . 15
References . . . 15

2 From Source to Executables: Towards Processes **16**
2.1 From source code to target-specific object code . . . 16
2.2 Why object files still need a linker . . . 17
2.3 From the stored-program idea to a concrete memory layout . . . 18
2.4 Linking produces the program QEMU can load . . . 19
2.5 Why execution starts before `main` . . . 19
2.6 The whole path, and what Make hides . . . 20
2.7 Change, rebuild, observe . . . 20
2.8 Why an executing program is not yet a process . . . 21
2.9 Yield, exit, resume, and restart . . . 22
2.10 The question that leads to execution context . . . 22
2.11 Key ideas . . . 23
Related Exercise . . . 24
References . . . 24

3 First Process: Stop and Continue **25**
3.1 C pointer notation: a reminder . . . 25
3.2 Building the process object . . . 26
3.2.1 Entry function . . . 26
3.2.2 Owned stack storage . . . 26
3.2.3 Lifecycle information . . . 27
3.2.4 What is still missing? . . . 27
3.3 Understanding the process stack . . . 28
3.4 Continuation: where should execution return? . . . 29
3.5 Context . . . 30
3.6 The complete process object . . . 30
3.7 The provided context-switch boundary . . . 31
3.8 Implementation boundary . . . 32
3.9 Public process interface . . . 33
3.10 The lifecycle in C . . . 33
3.10.1 Create / prepare . . . 33
3.10.2 Run and resume . . . 34
3.10.3 Yield . . . 35
3.10.4 Exit . . . 35
3.11 The complete CP1 observation . . . 35
3.12 Why CP1 deliberately uses yield . . . 36
3.13 Check your model . . . 36
3.14 Key ideas . . . 37
Related Exercise and Checkpoint . . . 37
References . . . 37

4 Where Our Code Runs **38**
4.1 Locate editing, building, and execution . . . 38
4.2 Establish a reliable edit–build–run loop . . . 39
4.3 One managed execution can contain several active calls . . . 40
4.4 Instructions and runtime data have different roles . . . 40
4.5 `sp`, `ra`, and `pc` are different kinds of CPU state . . . 41
4.6 Follow the same execution through yield and resume . . . 41
4.7 Connect the other memory regions . . . 42
4.8 The process stack is an array in an object . . . 43
4.9 Inspect the file without confusing it with live memory . . . 44
4.10 The next requirement: regain control without cooperation . . . 44
4.11 Key ideas . . . 45
Related Exercise and Handouts . . . 45
References . . . 45

Course Overview

This site contains the teaching material for **BVS1 – Betriebssysteme** at TH Köln, ICCT, for **Winter Semester 2026/2027 · TH Köln · ICCT**. The course is attended by students from multiple Bachelor programmes.

The course provides two aligned representations: concise lecture-specific Beamer slides for classroom interaction, and a reader-oriented chapter source used for both the PDF script and the HTML course.

Course Material

The lecture chapters are listed in the HTML navigation and in the PDF table of contents.

License

Except where otherwise noted, the student-facing BVS1 teaching materials are © 2026 Stavros Kousidis and licensed under Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0).

Third-party material remains subject to its respective license or copyright terms. The separately distributed BVS1 software repository `teaching-bvs1-code` remains licensed under the MIT License.

1 From Executing Code to Operating Systems

An operating system is familiar enough that it is easy to begin with examples rather than with the problem it solves. Windows, Ubuntu, Android and macOS are recognizable operating-system environments, but naming familiar systems does not yet explain why operating systems exist.

BVS1 therefore starts with a smaller question:

Can a computer execute a program without an operating system?

The first practical system is deliberately tiny. We use QEMU to provide a machine with an **RV64 (64-bit RISC-V)** CPU. This is our **target**: the machine for which we build the program. It runs one program and prints:

```
Hello BVS1
```

The experiment gives us a concrete machine to reason about before we introduce operating-system mechanisms.

1.1 One program can run without a guest operating system

At the lowest level, a processor executes instructions. Those instructions operate on registers and memory and can communicate with devices.

One important idea of the stored-program / Von Neumann model is that **instructions and data are both kept in memory**.

A deliberately simplified machine model is:

```
    instructions + data
           |
           ▼
        memory
           ▲
           |
           ▼
          CPU
           |
           ▼
```

```
        devices
```

This model omits many details of real hardware, but it is enough for the first distinction:

> **CPU, memory and devices exist independently of an operating system.**

The point here is conceptual rather than historical or microarchitectural. Modern processors may use separate instruction and data caches internally, but the important point here is that instructions and data both occupy target memory. Lecture 2 returns to this idea and makes the abstract memory model concrete.

In the `Hello BVS1` experiment, QEMU provides the target machine. The target contains an RV64 CPU, memory and devices. The program is loaded and begins executing without a guest operating system.

The physical computer still runs an operating system, and our Ubuntu development environment also runs an operating system. The important observation concerns the **target**:

> **Executing instructions is not, by itself, what makes an operating system necessary.**

1.1.1 Interpreting the experiment

The classroom experiment deliberately separates observation from explanation. Consider these statements:

- QEMU executed our program.
- A virtual CPU executed our program.
- Ubuntu executed our guest program.
- The target had an operating system.
- QEMU was already running before our guest program started.
- The string `Hello BVS1` existed somewhere in memory while the program was running.

The purpose of discussing them is to expose the execution model.

A precise description is:

- the Ubuntu development VM runs the QEMU process;
- QEMU implements the target machine;
- the target contains an emulated RV64 CPU, memory and devices;
- the RV64 program runs on that target;
- the target has no guest operating system.

Saying “QEMU executed the program” is understandable in informal language, but the more useful model is that QEMU **emulates the machine whose CPU executes the target instructions**.

1.2 The ISA connects software and processor

Machine code only has meaning if software and the processor agree on what each instruction means.

This agreement is the **Instruction Set Architecture (ISA)**.

An ISA specifies properties visible to machine-level software, including:

- instructions and their encodings,
- registers,
- memory-addressing rules,
- exceptional control transfers,
- privilege mechanisms.

Different processor implementations can implement the same ISA.

The ISA should therefore be distinguished from the **microarchitecture**:

- **ISA:** the contract visible to software;
- **microarchitecture:** how a particular processor implements that contract internally.

For BVS1, the teaching target uses **RV64**, the 64-bit form of RISC-V.

1.2.1 Why RV64?

RISC-V is a teaching choice, not a claim that it is the best ISA for every system.

It offers a comparatively regular architecture, public specifications, a modern privilege model, suitable development tools, and a well-supported QEMU `virt` machine. These properties let us expose operating-system mechanisms without making historical processor complexity the subject of the course.

1.3 Development host and target are different roles

The machine on which software is developed does not need to use the same ISA as the machine for which the program is built.

In BVS1:

- the **development host** is the Ubuntu `bvs1` VM in which the development tools run;
- the **target** is the RV64 machine for which we build the program.

The development VM may itself run on an x86-64 or Arm64 host platform. The teaching target is nevertheless RV64.

This gives two separate questions:

1. Where do the development tools execute?
2. Which machine will execute the generated program?

Keeping these roles separate will matter when we build code on one machine for another and when we examine emulation.

1.4 Virtualization and emulation are different

The most useful first question is:

Can the physical CPU execute the guest or target instructions directly?

That question separates the roles of virtualization and emulation more clearly than the word *virtual machine* by itself.

1.4.1 Virtualization: the guest instructions already belong to the host ISA

In hardware-assisted virtualization, guest and host normally use the same processor ISA.

A guest instruction is therefore, in the ordinary case, already an instruction that the physical CPU understands. Most non-privileged guest instructions can execute directly on the physical processor.

The virtualization mechanism controls the parts that must not simply be handed to the guest, for example:

- isolation between machines,
- privileged processor state,
- access to memory and devices,
- transitions that require intervention by the virtualization layer.

A simplified model is:

```
guest instruction
       ↓
physical CPU
```

The important point is not that virtualization performs no mediation. It is that the guest machine does not require a different instruction vocabulary from the physical CPU.

In BVS1, Multipass manages the Ubuntu development VM using virtualization facilities available on the host platform. The exact backend can differ between operating systems.

1.4.2 Emulation: the target instruction may not be a host instruction

Emulation can reproduce a machine whose ISA differs from the host ISA.

Suppose the physical processor uses x86-64 while the BVS1 target uses RV64. The physical x86-64 CPU does not directly understand an RV64 instruction.

An emulator must therefore reproduce the effect of the target instruction. Conceptually:

```
RV64 target instruction
          ↓
        QEMU
          ↓
interpret / translate
          ↓
host operations
          ↓
physical CPU
```

The word **interpret** is useful for the conceptual distinction, but it should not be understood as a claim that an emulator must process every instruction individually. QEMU can dynamically translate blocks of target instructions into host code.

The essential point is:

> **Virtualization can largely run guest instructions directly; emulation must reproduce what target instructions would do when the host cannot execute them directly.**

1.4.3 Our BVS1 machine stack

The resulting stack is:

```
physical host
     ↓  virtualization (managed through Multipass)
Ubuntu development VM: bvs1
     ↓  runs
QEMU
     ↓  emulates RV64
RV64 target machine
     ↓  executes
Hello BVS1
```

The layers have different roles:

- the **physical host** provides the real CPU, memory and devices;

- the **Ubuntu development VM** is the common environment in which the course tools run;
- **Multipass** manages that development VM and is not an additional machine layer;
- **QEMU** runs as a normal program inside the development VM;
- QEMU emulates the **RV64 target machine**;
- `Hello BVS1` executes on that target.

Only the RV64 target is the teaching machine whose operating-system mechanisms we will build. It initially contains no guest operating system.

A **virtual machine** is the machine environment presented to software. Virtualization and emulation are different techniques that can provide such environments.

QEMU itself is capable of more than the configuration used here: in other settings it can also participate in hardware-assisted virtualization. In BVS1, its relevant role is RV64 emulation.

1.5 The toolchain connects development host and target

The source program is written in a human-readable programming language, but the target CPU needs RV64 machine instructions.

The **toolchain** is the set of cooperating build tools. The compiler and assembler translate source into target machine code. They produce **object files**: target-specific intermediate files that still need to be combined. The **linker** combines those objects and resolves references between them.

These build products use **ELF (Executable and Linkable Format)**, a binary format used here for both object files and the final executable.

A simplified build path is:

```
source code
    ↓
cross compiler / assembler
    ↓
object files
    ↓
linker
    ↓
RV64 executable
    ↓
QEMU
    ↓
RV64 target machine
```

A **cross compiler** runs on one architecture while producing code for another target architecture. This is **cross-compilation**.

QEMU has a different role: it does not translate source code into the executable. It provides the machine on which the already generated target program can run.

1.5.1 Object files are not yet the complete program

Compilation and assembly can already produce target-specific object files. These can contain RV64 machine code, but they are not yet the complete executable. Separate pieces still have to be combined and references between them resolved.

The final result of this build path is a linked RV64 ELF executable that QEMU can load for the target machine.

Lecture 2 unfolds this path manually: we inspect the object files, discover what is still unresolved, and then see why linking and the target memory layout are necessary.

1.6 When does an operating system become useful?

The `Hello BVS1` target can run one tiny program without a guest operating system.

Now change the requirement:

> **The machine must run two programs.**

New questions immediately appear:

- Which program receives the processor?
- Where is each program's state kept?
- What happens when one program waits for a device?
- Can one program overwrite another program's memory?
- Who coordinates access to shared devices?
- How does a program request a shared service?

Increase the requirements further:

- many programs,
- multiple users,
- persistent storage,
- keyboard and display,
- networking,
- failures,
- programs that do not trust one another.

The problem is no longer merely “execute instructions”.

Now the system needs mechanisms for **sharing, coordination, protection and abstraction**.

This gives us the working definition used in BVS1:

> An **operating system** is software that helps turn hardware into a usable computer system by providing abstractions, resource management, coordination, protection, and services for programs and users.

This is intentionally a working definition.

We will test and sharpen it as concrete mechanisms are introduced.

Two classical viewpoints are already visible:

1. **Operating system as resource manager**
   CPU time, memory, storage and devices are finite resources that must be shared and controlled.
2. **Operating system as abstract machine**
   Programs should not need to manipulate every hardware detail directly. The operating system can provide safer and more useful abstractions and services.

1.7 Applying the working definition

The opening lecture deliberately asks students to classify several familiar terms before the working definition is established.

After deriving the definition, the distinctions become clearer:

- **Windows, Ubuntu and Android** are complete operating-system environments.
- The **Linux kernel** is the core used by Linux-based operating-system environments, not such an environment by itself.
- **UEFI** is firmware and a firmware interface/environment.
- **QEMU** is an emulator and virtualizer.
- **Docker** provides container tooling built on operating-system mechanisms.
- **Firefox** is an application.

The purpose of the exercise is not to memorize labels.

It is to ask which responsibilities a component actually assumes and at which layer it operates.

1.8 From Hello BVS1 to a minimal operating system

The course deliberately starts with a target that can do very little:

```
Hello BVS1
```

The semester question is:

> **What must we add to Hello BVS1 before we would call the result an operating system?**

Initial answers may include:

- handling devices,
- multiple executions,
- scheduling,
- protection,
- memory management,
- system services,
- communication,
- persistent storage,
- users,
- a command interface.

At the beginning these are hypotheses, not a checklist to memorize.

We add mechanisms only when a new requirement gives us a reason for them.

The practical target remains:

> **a small educational operating system on RV64 and QEMU `virt`, eventually providing a simple shell**

The red thread is:

```
a machine executes code
    ↓
software communicates with devices
    ↓
events must be handled
    ↓
multiple programs must coexist
    ↓
programs compete for the CPU
    ↓
programs and resources need protection
    ↓
```

```
programs need operating-system services
    ↓
programs need coordination
    ↓
programs need communication
    ↓
a user needs to control programs
    ↓
a simple shell
```

The course is therefore not organized as a catalogue of terminology.

Each new mechanism should answer a problem that the current system has acquired.

1.9 Key ideas

Lecture 1 establishes a small number of distinctions that we will use repeatedly:

- A target processor can execute a program without a guest operating system.
- CPU, memory and devices exist independently of the OS.
- An ISA defines the machine-level contract on which software relies.
- Development host and target are different roles.
- Virtualization and emulation are different techniques.
- Multipass gives us the virtualized development environment; QEMU emulates our RV64 target in the BVS1 setup.
- A cross-toolchain produces machine code for a target ISA different from the development system.
- Operating systems become useful as requirements for sharing, abstraction, coordination, protection and services grow.
- The BVS1 system grows only when a new requirement motivates a new mechanism.

A useful test of understanding is to explain this complete path:

```
source code
    ↓
cross-toolchain
    ↓
RV64 executable
    ↓
QEMU program in the development VM
    ↓
emulated RV64 machine
    ↓
observable program behaviour
```

and then explain why successful execution still does **not** imply that the target contains an operating system.

Related Exercise

Exercise 1 — From Source to First Instructions also consolidates concepts from this lecture, especially the development-host/target distinction and virtualization versus emulation. Its main placement is after Lecture 2, where it also applies the toolchain and startup material.

References

- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. https://pages.cs.wisc.edu/~remzi/OSTEP/
- Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, 5th ed., Pearson, 2023. ISBN 978-0-13-761888-0. https://www.pearson.com/en-us/subject-catalog/p/modern-operating-systems/P200000003295/9780137618880
- RISC-V International, *RISC-V Ratified Specifications Library*. https://docs.riscv.org/
- QEMU Project, *RISC-V `virt` Generic Virtual Platform*. https://www.qemu.org/docs/master/system/riscv/virt.html
- Canonical, *Multipass Documentation*. https://documentation.ubuntu.com/multipass/

2 From Source to Executables: Towards Processes

Lecture 1 showed that an RV64 target can execute a small program without a guest operating system. Lecture 2 asks how that execution comes into existence and why, for an operating system, simply starting a program is not enough.

The path is:

```
source → objects → linked ELF → memory → startup code → main
```

and then:

```
executing program → OS-managed execution → process
```

The build tools are therefore not the final topic. We unfold them once because they make the transition from source code to a running machine state visible.

2.1 From source code to target-specific object code

We first compile `main.c` without linking:

```
mkdir -p build/manual

riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -msmall-data-limit=0 \
  -c main.c \
  -o build/manual/main.o
```

The important option is `-c`: stop before linking. The remaining options are course-provided target/build settings.

Inspecting the result with

```
file build/manual/main.o
```

shows that `main.o` is already a RISC-V ELF object.

ELF (**Executable and Linkable Format**) is the binary file format we use here for both object files and the final executable.

This distinction matters: the compiler has already translated C into code for our target ISA. The object file is therefore not architecture-neutral. But it is also not yet a complete program. Separate pieces can still refer to one another, and their final addresses have not yet been established.

The startup assembly is translated in the same way:

```
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -c start.S -o build/manual/start.o
```

2.2 Why object files still need a linker

A **symbol** is a name used to refer to a program entity or location, such as a function or a linker-defined address.

Using `nm` ("name list") lets us inspect the symbols in the two objects:

```
riscv64-unknown-elf-nm build/manual/start.o
riscv64-unknown-elf-nm build/manual/main.o
```

For the symbols relevant here, we can read the result as:

```
start.o defines _start
start.o needs   main
start.o needs   _stack_top

main.o  defines main
```

The important concept is not the letters printed by `nm`, but the relationship between separately translated pieces. One object can provide a symbol while another refers to it. The linker connects those pieces.

The unresolved `_stack_top` also reveals a second responsibility of the linker: it must establish where the program lives in target memory.

2.3 From the stored-program idea to a concrete memory layout

Lecture 1 used the abstract model:

```
instructions + data
        ↓
      memory
        ↕
       CPU
```

Our real program now contains different kinds of bytes that need concrete places.

The **stack** is runtime memory used during function calls, for example for local state and saved call information. The register `sp` is the stack pointer; `_stack_top` names the initial top address of this stack.

```
RV64 memory from 0x80000000

+-----------------------------------------------------------+
| .text    _start, main, ...      | instructions          |
| .rodata  "Hello BVS1\n"         | constant data         |
| .data                           | initialized data      |
| .bss                            | zero-initialized data |
| C stack  (top: _stack_top)      | runtime data          |
+-----------------------------------------------------------+
```

These roles are not properties that memory itself knows. They arise from the executable layout and from how the program and CPU use the addresses.

The course-provided `linker.ld` makes that layout concrete:

```
ENTRY(_start)

MEMORY {
    RAM (rwx) : ORIGIN = 0x80000000, LENGTH = 128M
}

. = ALIGN(16);
. += 16K;
_stack_top = .;
```

Here `.` is the linker's current output address. After 16 KiB have been reserved for the initial stack, that address is named `_stack_top`.

Students do not need to write linker scripts in BVS1. The important idea is that the linker does not merely join files; it constructs one final arrangement of addresses in target memory in which the pieces can refer to one another.

2.4 Linking produces the program QEMU can load

We link the two objects using the course layout:

```
riscv64-unknown-elf-gcc -nostdlib -nostartfiles -no-pie \
  -T linker.ld build/manual/start.o build/manual/main.o \
  -o build/manual/hello-bvs1.elf
```

Here `-T linker.ld` selects our course-provided memory layout; the remaining options are link settings for our target without a guest operating system.

At this point the linker has:

- combined the objects;
- resolved the symbol references;
- established the final memory layout.

The resulting ELF is what QEMU receives. QEMU does not compile or link the program; it provides the emulated RV64 machine and loads the already-built image.

2.5 Why execution starts before `main`

The linker script declares:

```
ENTRY(_start)
```

and `_start` is defined in `start.S`:

```
_start:
    la   sp, _stack_top
    call main
1:
    j    1b
```

This small fragment exposes an important boundary between machine startup and ordinary C code.

Before `main` can run, startup code establishes at least the execution environment that C expects. Here that means setting `sp`, the stack pointer, to the address denoted by `_stack_top`.

Then `call main` transfers control to `main` and remembers where execution should return. We deliberately do not open the full function-call mechanism yet. The fact that a return point must somehow be preserved becomes important in Lecture 3.

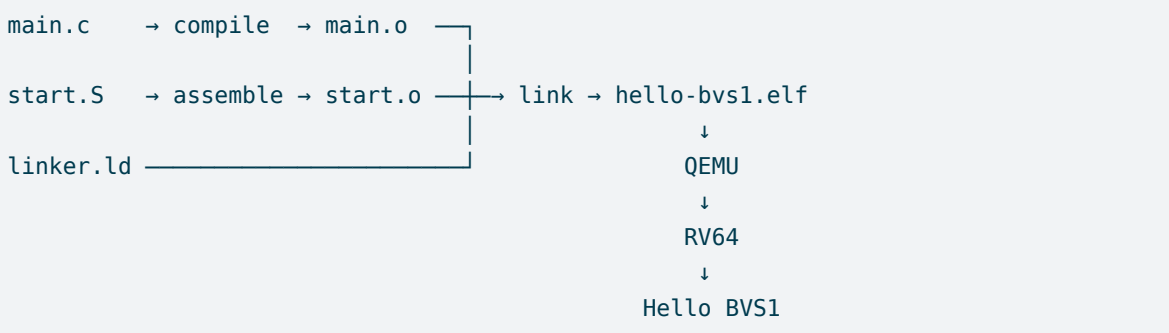
The local label `1:` and `1b` only express the final fallback loop: if `main` returns, execution remains there.

2.6 The whole path, and what Make hides

Running the ELF manually:

```
qemu-system-riscv64 -machine virt -m 128M \
  -smp 1 -nographic -bios none \
  -kernel build/manual/hello-bvs1.elf
```

reconstructs the complete path:



Compilation, assembly, linking, and QEMU itself run in the Ubuntu development VM. The final RV64 instructions execute in the emulated target.

Make automates this workflow. It decides when a target is out of date and runs the corresponding recipe. The GCC driver performs the translation steps and invokes the linker. QEMU provides the machine on which the result executes.

Doing the path manually once is useful because the automation then has a visible meaning rather than being a black box.

2.7 Change, rebuild, observe

If only the string in `main.c` changes, we can predict the consequences before rebuilding:

- `main.o` must change;
- `start.o` does not;
- the final ELF must be linked again;
- `linker.ld` does not change;
- QEMU itself does not change.

These are the logical dependencies. A concrete Makefile may encode them at a coarser granularity and therefore rebuild more than the minimum set of intermediate files.

This small exercise is about maintaining a causal model of the system:

predict → change → build → run → observe → explain

That pattern will be reused when the changes are no longer only strings, but operating-system mechanisms.

2.8 Why an executing program is not yet a process

Our current target has one execution:

```
_start
  ↓
main
  ↓
print
  ↓
for (;;) {}
```

Once it starts, nothing inside our target can currently stop it, regain control, and later continue it.

That is the new requirement. An operating system must be able to manage an execution rather than merely start it.

For our first model:

> A **process** is an execution that the operating system can manage.

For Checkpoint 1, "manage" means only:

```
create → run → yield → resume → exit
```

This is intentionally much smaller than a complete Unix process model. We do not yet need address spaces, files, process IDs, or scheduling policies.

The minimum lifecycle is:

```
create ──→ READY ──run / resume──→ RUNNING ──exit──→ FINISHED
             ↑                        │
             └──────── yield ─────────┘
```

A yielded process is still runnable. A finished process is not.

2.9 Yield, exit, resume, and restart

Yield and exit both transfer control back to the operating system, but they have different meanings.

- **yield**: the execution is not finished and must be able to continue later;
- **exit**: the execution is finished and must never continue.

The distinction becomes visible in this intended interface:

```
uart_puts("A\n");
process_yield();
uart_puts("B\n");
process_exit();
```

`process_yield()` and `process_exit()` are not library functions we already have. They are functions our minimal operating system still has to implement.

After `process_yield()`, the operating system has control. When the process runs again, it must continue with `uart_puts("B\n")`. Printing `A` again would mean that we restarted the execution instead of resuming it.

This is the conceptual step that makes the next lecture necessary.

2.10 The question that leads to execution context

To resume correctly, something about the paused execution must survive:

```
process runs
     ↓
yield
     ↓
OS regains control
     ↓
resume
     ↓
continue exactly where it stopped
```

What must be preserved so this can work?

At this point it is enough to suspect that some machine state must survive the pause. Lecture 3 determines exactly what that state is.

The final path of Lecture 2 is therefore:

```
source → objects → linked ELF → memory → _start → main
```

followed by:

```
executing program → OS-managed execution → process
```

The remaining question is:

> **What machine state makes resume possible?**

2.11 Key ideas

Lecture 2 connects the build path to the first process abstraction:

- Compilation and assembly produce target-specific object files.
- ELF is the binary format we use for both object files and the final executable.
- Symbols name program entities or locations; the linker resolves references between separately translated pieces.
- The linker also establishes the final memory layout of the program.
- The stack is runtime memory used during function calls, and `sp` is the stack pointer.
- Execution begins in startup code at `_start`, which establishes the initial environment before entering C.
- QEMU loads the already-built ELF into the emulated RV64 target; it does not compile or link the program.
- Make automates the build workflow, while the compiler, assembler, and linker perform the translation steps.
- A process is an execution that the operating system can manage through `create → run → yield → resume → exit`.
- `yield` and `exit` both return control to the operating system, but only a yielded execution may continue later.
- Resuming means continuing where execution stopped, not restarting from the beginning.

A useful test of understanding is to explain both chains:

```
source → object files → linked ELF → memory → _start → main
```

and

```
executing program → OS-managed execution → process
```

and then explain why resuming a process requires some machine state to survive the pause.

Related Exercise

Exercise 1 — From Source to First Instructions is the exercise associated with this lecture. It reconstructs the host/target layers, follows the source-to-executable path, inspects symbols and startup code, and prepares the transition toward the first process.

References

- RISC-V International, *RISC-V ABIs Specification*. https://riscv-non-isa.github.io/riscv-elf-psabi-doc/
- GNU Compiler Collection, *RISC-V Options*. https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html
- GNU Binutils, *The GNU linker (ld)*, especially the sections on linker scripts, symbols, `MEMORY`, and `SECTIONS`. https://sourceware.org/binutils/docs/ld.html
- GNU Binutils, *nm*. https://sourceware.org/binutils/docs/binutils/nm.html
- QEMU documentation, *RISC-V System Emulator*. https://www.qemu.org/docs/master/system/target-riscv.html
- QEMU documentation, *`virt` Generic Virtual Platform*. https://www.qemu.org/docs/master/system/riscv/virt.html
- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, especially the process chapters. https://pages.cs.wisc.edu/~remzi/OSTEP/

3 First Process: Stop and Continue

Lecture 2 ended with a requirement: run an execution, regain control, and continue that same execution later. Two questions follow: **what must survive the pause, and how do we build that?**

Our working definition is:

> **A process is an execution that can be managed.**

In BVS1, our minimal operating system will do the managing. For our first process, managing means:

```
create -> run -> yield -> resume -> exit
```

This is the simplified lifecycle used for Checkpoint 1, rather than a general definition of every process's lifetime. To stop and continue the same execution, our process needs an entry function, its own stack, saved CPU register state (its context), and lifecycle information: state and exit status.

This chapter builds those parts into one resumable execution. Creation prepares it without running its code; run starts or resumes it; yield returns control while preserving the ability to continue; exit finishes it.

3.1 C pointer notation: a reminder

The process representation will be a C struct. These expressions distinguish an object, its address, and a pointer to it:

```
struct process p;
struct process *q = &p;
```

Here `p` is the object, `&p` is its address, and `q` stores that address. In the declaration, `*` makes `q` a pointer; in an expression, `*q` dereferences the pointer to access the object.

`p.state` accesses the object's state directly. `(*q).state` accesses the same member through the pointer; `q->state` is shorthand for that expression. The same notation lets us access a nested member such as `q->context.sp` once the context has been added.

For function pointers, storing a function and calling it are different operations:

```
p.entry = worker;
p.entry();
```

In the assignment, `worker` converts to a pointer to that function, so the first line stores where to call. It does not call `worker`. The second line calls it through the stored pointer; `worker()` would call the function directly.

3.2 Building the process object

We first give the execution a concrete representation. The following struct fragments add one part at a time; the complete definition follows after we establish what must be saved in a context.

3.2.1 Entry function

An entry function is where a new process begins its work. For example:

```
void worker(void) { /* process-specific work */ }
```

The process must remember which function to enter. Its first field is therefore a function pointer:

```
struct process {
    void (*entry)(void);
};
```

Read `void (*entry)(void)` as: `entry` is a pointer to a function taking no arguments and returning `void`. Repeating that declaration is awkward, so a `typedef` gives the pointer type a name:

```
typedef void (*process_entry_t)(void);
```

The field can now be written `process_entry_t entry;`. This names the same function-pointer type; it does not declare or call an entry function.

3.2.2 Owned stack storage

The process also reserves memory for its own call stack:

```c
#include <stdint.h>

#define STACK_SIZE 4096

struct process {
    process_entry_t entry;
    uint8_t stack[STACK_SIZE];
};
```

`uint8_t`, defined by `<stdint.h>`, is an unsigned 8-bit integer type. A `char stack[STACK_SIZE]` array would also be a viable choice for raw byte storage in our RV64 environment: a character array need not contain text. We use `uint8_t` to make the byte-storage role explicit and keep our treatment of integer types focused on `<stdint.h>`. This header is available in our freestanding RISC-V toolchain; including it does not require linking a C library.

At this point, treat the array simply as memory owned by the process; we will explain its use below.

3.2.3 Lifecycle information

The manager needs to know whether the process may run and, once it finishes, which exit status it produced:

```c
enum process_state { READY, RUNNING, FINISHED };

struct process {
    process_entry_t entry;
    uint8_t stack[STACK_SIZE];
    enum process_state state;
    int exit_status;
};
```

An `enum` supplies named alternatives for the state. READY means the process may run, RUNNING means it is executing, and FINISHED means it must not run again. `exit_status` records the result once it finishes.

3.2.4 What is still missing?

We can now remember where to start, where call data can live, whether the process may run, and how it finished. But suppose execution pauses here:

```
helper();
process_yield();
/* continue here later */
```

These fields do not yet tell us how to continue at exactly that point. We need to understand both the stack's runtime data and the CPU register values that make that data usable.

3.3 Understanding the process stack

The process needs **owned stack storage**: memory that remains associated with the process while the operating system executes elsewhere.

The process's machine instructions are **not** copied onto that stack. They remain in the program's `.text` section. The stack contains runtime data associated with active calls—for example saved register values, return information, some local variables, and temporary values when needed. Not every local variable is stored on the stack; the compiler may keep values in registers.

The array reserves storage; `sp`, the CPU's stack-pointer register, selects the current position in that storage. Array indices increase toward higher addresses, while the RV64 C stack grows toward lower addresses:

```
                  stack grows this way
                        <-----

lower addresses                            higher addresses
&stack[0]               sp                &stack[STACK_SIZE]
    v                   v                             v
    +---------------------------------------------+
    |                   | -- used stack ------------|
    +---------------------------------------------+
```

`&stack[STACK_SIZE]` is the address just past the array's last byte. A fresh process starts with an appropriately aligned `sp` at the high end; stack use then extends toward lower addresses. Reserving the array alone does not change the CPU's active stack pointer.

Nested calls belong to the same execution and use the same process stack. For example:

```
void helper(int *x) { (*x)++; }
void worker(void) { int n = 1; helper(&n); }
```

Conceptually, while `helper()` is active:

```
                          sp
lower addresses           v               higher addresses
    +-----------------------------------------------+
    |             | helper frame | worker frame | - |
    +-----------------------------------------------+
```

The `helper frame` and `worker frame` labels denote runtime data associated with the active calls. The stack does **not** contain the machine instructions of `helper()` or `worker()`: the CPU executes those instructions from the `.text` section of the program's memory layout.

The newer call uses space toward the lower-address end of the used stack. This is a schematic picture, not an exact compiler-generated layout: the compiler may inline calls or keep values in registers.

If the process yields from a nested call, that call chain's stack data must remain intact while the OS uses its own stack. Resuming requires the process's saved `sp` as well as the preserved storage.

3.4 Continuation: where should execution return?

Lecture 2 showed the startup fragment:

```
call main
```

A RISC-V `call` remembers the location to which execution should later return. That location is stored in `ra`, the **return-address register**.

An ordinary return therefore already contains a useful idea:

```
call function
    |
    v
function executes
    |
    v
ret
    |
    v
continue at address stored in ra
```

If a process yields and is later resumed, the operating system must preserve some representation of the place where execution should continue. Preserving `ra` makes that idea concrete.

The stack data must survive too, and the process must recover the stack position recorded in `sp`. Restoring a continuation with the OS stack would not continue the same call chain.

3.5 Context

At a pause, some execution state is still in CPU registers. Another execution can overwrite those values even when the process stack remains intact. The saved CPU register state needed to continue the execution is its **context**.

For the cooperative CP1 mechanism, the provided representation is:

```
struct context {
    uintptr_t ra;    // continuation address
    uintptr_t sp;    // current stack position
    uintptr_t s[12]; // saved s0 through s11
};
```

`uintptr_t`, defined by `<stdint.h>`, is an **unsigned integer type**, not a pointer type. It can hold the integer representation of a pointer value and later be converted back without losing that value. In our RV64 toolchain it is 64 bits wide. We use it for saved register-sized values such as `ra` and `sp`, which contain addresses during normal execution. Students are not expected to study the complete family of fixed-width integer types here.

The array `s` holds the RISC-V callee-saved registers `s0` through `s11`: `s[0]` stores `s0`, through `s[11]` storing `s11`. A **calling convention** is the agreement that lets separately compiled functions cooperate about how registers are used. A **callee-saved register** is one whose value a called function must preserve for its caller. Preserving those required registers allows an ordinary C function call to cross the context-switch boundary safely.

Students do not need to memorize that register list or derive the complete assembly routine from the ABI. For CP1 they should be able to explain why `ra` and `sp` matter and why some additional register state must also survive.

3.6 The complete process object

The saved context supplies the missing part. Together, entry, stack, context, and lifecycle information represent our managed execution.

The public representation used by Lecture 3, Exercise 2, and Checkpoint 1 is:

```
struct process {
    process_entry_t entry;
    uint8_t stack[STACK_SIZE];
    struct context context;
    enum process_state state;
    int exit_status;
};
```

The five fields have distinct roles:

- `entry`: where a new process begins;
- `stack`: process-owned raw byte storage for its call stack;
- `context`: saved CPU register state needed to continue execution;
- `state`: whether the process is READY, RUNNING, or FINISHED;
- `exit_status`: the result recorded when the process finishes.

Exercise 2 uses this exact field order because its precompiled process library was built against this representation. Checkpoint 1 keeps the same interface and representation while students provide the implementation.

3.7 The provided context-switch boundary

The course provides a function implemented in RISC-V assembly, with this C declaration:

```
void context_switch(
    struct context *current_context,
    const struct context *next_context);
```

`current_context` points to storage where the running execution's registers will be saved. `next_context` points to the saved registers to restore. The `const` qualifier says the function does not modify the context object through `next_context`. Either context may belong to the OS execution or the process execution.

A **context switch** in this part of BVS1 means:

1. save the current execution context;
2. restore another saved context;
3. continue the restored execution.

The assembly below shows the matching save and restore operations. The ellipses stand for the corresponding operations on `s1` through `s10`:

```
sd ra,    0(a0)
sd sp,    8(a0)
sd s0,   16(a0)
...
sd s11, 104(a0)

ld ra,    0(a1)
ld sp,    8(a1)
ld s0,   16(a1)
...
ld s11, 104(a1)
```

```
ret
```

`sd` (store doubleword) copies a 64-bit register value into memory; `ld` (load doubleword) copies a 64-bit value from memory into a register. In `sd s0, 16(a0)`, the destination is the address in `a0` plus 16 bytes. The offsets follow the context layout: `ra` at byte 0, `sp` at byte 8, and the twelve saved `s` registers at bytes 16 through 104.

After the loads, `ret` uses the newly restored `ra`. The switch therefore changes both the stack that is active and the location at which execution continues.

The stored fields and the restored fields match: `ra`, `sp`, and all twelve entries of `s`. The process needs both its stack memory and these saved register values to continue the same execution.

The function arguments `current_context` and `next_context` arrive in the RISC-V argument registers `a0` and `a1`. These registers are caller-saved: their previous values are not part of the state that an ordinary C call promises to preserve. In this cooperative switch they are used to locate the two context structures; the persistent saved context is `ra`, `sp`, and `s0-s11`.

The same operation works in both directions:

```
OS running                          process running
    |                                    |
    | save OS context                    | save process context
    | restore process context            | restore OS context
    v                                    v
process running                       OS running
```

There is no scheduler here. CP1 contains only one process.

3.8 Implementation boundary

The course provides the machinery whose complexity is incidental to the CP1 competence:

- existing startup and linker support;
- UART output already available in the starter;
- the context representation;
- the RV64 context-switch assembly boundary.

Students construct the process abstraction and lifecycle in C:

- process representation;
- owned stack;
- first-run context;

- entry handling;
- run;
- yield;
- exit;
- observable resume behaviour.

CP1 does **not** require timer interrupts, preemption, several runnable processes, a scheduler, privilege separation, virtual memory, dynamic loading, or dynamic allocation.

3.9 Public process interface

The first-process abstraction uses this public C interface:

```
void process_create(struct process *process,
                    process_entry_t entry);
void process_run(struct process *process);
void process_yield(void);
void process_exit(int status);
```

Exercise 2 links this interface against a provided implementation. Checkpoint 1 keeps the same interface but replaces that implementation with student-written C code.

3.10 The lifecycle in C

Lecture 2 introduced:

```
create -> run -> yield -> resume -> exit
```

Lecture 3 now connects each semantic step to the minimum implementation work.

3.10.1 Create / prepare

Preparing a process means constructing a valid first-run state:

```
remember entry function
        |
choose process stack top
        |
prepare initial context
        |
state = READY
```

No process code has executed yet.

A process that has already yielded has a saved continuation. A new process does not.

The restore mechanism nevertheless expects a context containing a stack pointer and a continuation. The solution is to prepare an initial context before the process first runs.

Its initial stack pointer is placed at the aligned high end of its owned stack array.

The initial continuation is a small **bootstrap function**, called `process_start()`: an internal function used to enter a process for the first time. It is an implementation detail, not another public process operation.

Conceptually:

```
initial sp -> aligned high end of process.stack
initial ra -> process_start -> process.entry()
```

The first restore followed by `ret` therefore enters `process_start()` while using the process's own stack.

`process_start()` calls the process entry function. If that function returns normally, `process_start()` calls `process_exit(0)`. Returning from the entry function therefore finishes the process with status 0 and transfers control back to the OS.

Later resumes do not use the prepared bootstrap address again. A yield saves the real continuation from the running process, so the next restore continues after the context-switch call inside the yield path.

This is an important simplification:

First start and later resume use the same low-level restore mechanism.

3.10.2 Run and resume

The OS keeps a pointer named `current_process` to identify the process whose execution currently owns the CPU. `process_yield()` and `process_exit()` take no process argument: they use this pointer to find the object whose state and context they must update.

`process_run(p)` performs these steps:

```
reject p if its state is FINISHED
set current_process to p
set p's state to RUNNING
context switch from OS to p
```

The FINISHED check belongs to `process_run()`: a saved context does not by itself grant permission to run again.

The same run operation can support two cases:

- a READY process that has never run starts through its prepared bootstrap context;
- a READY process that yielded earlier resumes through the context saved by that yield.

The required behaviour is **resume**. A separate `process_resume()` API is not necessary.

3.10.3 Yield

For CP1:

> **yield** means that the running process voluntarily returns control to the operating system while remaining able to resume later.

Using `current_process`, yield marks the process READY, saves its current context, and restores the OS context.

When the operating system later runs that process again, execution continues after the context-switch call inside the yield path.

3.10.4 Exit

For CP1:

> **exit** means that the process finishes, records a result, and returns control to the operating system.

Using `current_process`, exit records the status in the process object, marks it FINISHED, and switches from the process context back to the OS context.

A finished process must not be resumed.

Yield and exit can use the same low-level control-transfer boundary, but their lifecycle meaning is different.

3.11 The complete CP1 observation

Exercise 2 and Checkpoint 1 use the same small permutation program. Its observable behaviour is:

```
process: before yield: VBS1
os: process yielded
process: result: VB1S
os: process exited
```

The interesting evidence is not the strings themselves. The output demonstrates that:

1. the process starts using its own execution context;
2. `worker()` calls `permute()`, which modifies process-local data and yields before returning;
3. the operating system regains control while that call chain remains suspended;
4. the same `permute()` invocation later continues after `process_yield()`;
5. the local array progresses from `VBS1` to `VB1S`, showing that the same execution state survived;
6. the process exits with status 0 and becomes FINISHED;
7. the operating system regains control again and must not resume the finished process.

3.12 Why CP1 deliberately uses yield

CP1 needs a control transfer that gives the operating system control **without finishing the process**.

A timer interrupt would already introduce the preemption mechanism needed later for CP2. Blocking or sleeping would require an additional condition that makes the process runnable again. An ordinary C return would not expose the requirement to preserve and later restore a continuation.

A cooperative yield is therefore the smallest mechanism that makes resumability observable.

It also creates the next operating-system problem cleanly:

> What if a process never yields?

That question motivates timer-driven preemption on the way to CP2.

3.13 Check your model

Use these questions to check the distinction between restarting, resuming, and finishing:

1. What breaks if we call the entry function again after yield instead of restoring the saved context?
2. What breaks if we restore `ra` but not `sp` and `s0-s11`?
3. What breaks if `process_run()` allows a FINISHED process to execute again?

A successful explanation should connect each failure to the process representation: calling entry again restarts its work, restoring only part of the context cannot preserve the same execution, and restoring a finished process violates its lifecycle state.

3.14 Key ideas

- A **process** is a managed execution.
- Our process representation combines an entry function, owned stack storage, CPU context, lifecycle state, and exit status.
- The **stack** is process-owned memory for the runtime data of active function calls. Function instructions remain in `.text`.
- A **context** is the saved CPU register state needed to continue the same execution.
- **Lifecycle state** says whether an execution may run; **context** says how it continues.
- A **context switch** saves the current CPU register state and restores the next one. The same mechanism transfers control in both directions between OS and process.

Related Exercise and Checkpoint

Exercise 2 — Using Your First Process lets you use the process abstraction before implementing it yourself.

Checkpoint 1 — I Have a Process is the corresponding implementation milestone.

References

- RISC-V International, *RISC-V ABIs Specification*. https://riscv-non-isa.github.io/riscv-elf-psabi-doc/
- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, especially the process chapters. https://pages.cs.wisc.edu/~remzi/OSTEP/
- Russ Cox, Frans Kaashoek, Robert Morris et al., *xv6: a simple, Unix-like teaching operating system* and xv6-riscv source, used here only as a comparative reference for low-level context switching. https://pdos.csail.mit.edu/6.S081/2020/xv6/book-riscv-rev1.pdf

4 Where Our Code Runs

Lecture 3 constructed a process from an entry function, owned stack storage, saved CPU context, and lifecycle information. This chapter connects that abstraction to the machines and memory we actually use. It follows four questions raised by students: Where am I working? How do I edit the files that are actually built? Where is the program counter? Where do constants appear in the disassembly? Together they lead to the guiding question: **where are our instructions and runtime data, and what must be preserved when execution pauses or is interrupted?**

We use the same `worker()` and `permute()` from Exercise 2. The aim is to reconstruct and diagnose the existing mechanism before extending it.

4.1 Locate editing, building, and execution

Our setup has three machine roles:

```
physical host: Linux / macOS / Windows
    |  hosts the VM; Multipass manages it
    v
Ubuntu VM bvs1: our development host
    |  runs editor, build tools, and QEMU
    v
QEMU emulates an RV64 target: CPU + RAM + UART
    |  target CPU executes RV64 instructions
    v
our program / growing BVS1 operating system
```

The physical host supplies the outer infrastructure. Multipass manages the Ubuntu VM; it is not another machine layer. The VM is the **development host** for our tools. QEMU runs there as an ordinary Ubuntu process and emulates the **target machine**.

Both our OS-side control code and the managed process execute on the RV64 target. They do not become Ubuntu programs merely because QEMU runs in Ubuntu. The cross compiler and linker execute in the VM but generate an RV64 ELF for the target ISA.

The complete workflow is:

```
edit -> save source -> compile/link -> linked ELF
                                            |
                                     QEMU loads the image
                                            |
                                    RV64 instructions execute
```

Make automates the build recipe. The editor changes source text; the compiler reads the saved file; QEMU receives the resulting ELF. An already running QEMU instance does not automatically acquire a new image when source files are edited or rebuilt.

4.2 Establish a reliable edit–build–run loop

Start on the physical host with:

```
multipass shell bvs1
```

Inside the VM, locate the existing repository and source file:

```
cd ~/teaching-bvs1-code
pwd
ls
nano main.c
```

Run `make` from the repository directory that contains the `Makefile`. Make reads the `Makefile` in the current directory, and that `Makefile` names the source files it compiles. A `main.c` edited in another directory or another copy of the repository is not this build's input.

In nano, save with **Ctrl-O**, **Enter**, then leave with **Ctrl-X**. If using Vim, enter Insert mode with **i**, make the change, press **Esc**, then save and leave with `:wq`. The editor handout contains further details. The relevant competence is editing the intended file reliably; the choice of editor is not assessed.

After saving, inspect your change and build:

```
git diff -- main.c
make
make run
```

Use one small output-label change to connect the saved file to an observable effect. The array's permutation should remain unchanged if only a label changes. End QEMU with **Ctrl-a**, release, then **x**. Restore that intentional label change before continuing.

If an old label appears, diagnose the chain in order:

```
saved source -> build input -> successful build -> executable -> QEMU run
```

Was the change saved? Is the saved file the one Make uses as build input? Did the build succeed and produce a new executable? Did a new QEMU session load that executable? Rebuilding repeatedly cannot correct an unsaved buffer or a different repository copy.

4.3 One managed execution can contain several active calls

Exercise 2 uses this entry function:

```
static void worker(void)
{
    char value[] = "BVS1";
    permute(value);
    uart_puts("process: result: ");
    uart_puts(value);
    uart_puts("\n");
    process_exit(0);
}
```

The local array has five bytes: `B`, `V`, `S`, `1`, and the terminating zero byte. It is modifiable character data. The helper receives a pointer to that existing array; passing it does not create another array or another process.

The helper first exchanges positions 0 and 1, prints the intermediate value, calls `process_yield()`, then exchanges positions 2 and 3. Before yield, both `worker()` and `permute()` are still active. The array has become `VBS1\0`, but its owning worker invocation has not ended.

4.4 Instructions and runtime data have different roles

The linked program places machine instructions in `.text`. This includes instructions implementing the worker, helper, process API, and OS-side code. Calling a function transfers control to its instructions; it does not copy those instructions onto the stack.

The **program counter**, `pc`, identifies the instruction to execute. The **stack pointer**, `sp`, selects the current stack position. These are CPU state, not memory sections. A saved return address is an address value that points back into code; its being stored on the stack does not make the pointed-to instructions stack contents.

The stack holds runtime data associated with active calls: saved call information, saved registers, some local variables, and temporaries as needed. The labels `worker frame` and `permute frame` describe this runtime data, not the functions' instructions.

A schematic snapshot after the first swap is:

```
.text                       process-owned stack storage
                            higher addresses
same worker instructions    +-------------------------+
same permute instructions   | worker: value V B S 1 \0 |
  ...                       +-------------------------+
  call process_yield        | permute call data       |
                            +-------------------------+
                            lower addresses
```

The CPU has changed data in the array while executing instructions that remain in `.text`. A non-inlined helper call uses the same process stack as the worker, with newer call data toward lower addresses. The picture does not prescribe exact offsets, compiler-generated frames, or a stack slot for every local value.

4.5 `sp`, `ra`, and `pc` are different kinds of CPU state

`sp` and `ra` are the conventional ABI names of the integer registers `x2` and `x1`. Ordinary instructions read and write them like the other general-purpose registers `x0`–`x31`. The program counter is also architectural CPU state, but it is not one of these 32 registers. The CPU advances it as it executes, and jumps, branches, calls, and returns change it.

This explains why the Lecture 3 context contains `ra`, `sp`, and `s0`–`s11` but no explicit `pc`. The cooperative context switch happens at an ordinary function-call boundary, so the saved `ra` already describes the continuation:

```
saved ra -> restored ra -> ret -> execution continues there
```

`ret` sets the program counter to the restored `ra`. The program counter is therefore not forgotten; the cooperative switch re-establishes it through `ra`.

4.6 Follow the same execution through yield and resume

When the process yields, the low-level switch saves its continuation, stack position, and required additional registers. The OS context is restored. The CPU's current `pc` and `sp` now describe OS execution, while `p.context` holds the saved register values for the paused process. The array and suspended call data remain in `p.stack`.

The next `process_run()` restores that saved context. Low-level continuation first returns through the suspended yield path. `process_yield()` then returns to the same `permute()` invocation, which performs the second swap. The array becomes `VB1S\0`. The worker does not initialize it again, because no new worker invocation was started.

When the helper returns, the worker prints the result and calls `process_exit(0)`. Exit marks the process FINISHED and returns control to the OS. Old bytes may physically remain after a helper returns or a process exits; that does not keep a call active or authorize another run. Lifecycle state determines whether the process may run again.

Moment	Array contents	CPU executes
worker initialized it	`BVS1\0`	process
first swap completed	`VBS1\0`	process
yielded	`VBS1\0`, preserved	OS
second swap completed	`VB1S\0`	process
process exited	no active array	OS

Calling entry again restarts the work; restoring a continuation without the matching stack position loses the original call chain; allowing a FINISHED process to run violates its lifecycle. These are different failures of the same model.

4.7 Connect the other memory regions

The section names describe roles in the linked image:

Region	Role in our example
`.text`	RV64 machine instructions
`.rodata`	constant data, including output labels
`.data`	writable static/global objects with stored initial values, when present
`.bss`	zero-initialized static/global storage, including OS bookkeeping in the inspected library
OS stack	runtime data of OS-side calls
process stack storage	runtime data of the managed process's calls

The output label `"process: result: "` is constant data, normally placed in `.rodata`. The initialized local `value` array is a separate writable object. Its C initializer does not put that array in `.rodata`: the compiler can provide its initial bytes from constants in instructions or from constant storage. Inspect the actual build rather than assuming a separate `BVS1` string must be present in `.rodata`.

Constants therefore appear in the disassembly in two forms:

- A value small enough to encode inside an instruction appears as an **immediate** operand, for example in `li` or `addi`. Larger values may be built by more than one instruction. The four characters of `BVS1` can appear together as one 32-bit value: our

target uses little-endian byte order, so storing `0x31535642` produces the bytes `0x42 0x56 0x53 0x31` at increasing addresses.
- In this build, constant output labels are stored in `.rodata`. Instructions compute a label's address, typically with an `auipc`/`addi` pair that `objdump -d` annotates with the resulting address, and pass it to `uart_puts()`. `riscv64-unknown-elf-objdump -s -j .rodata build/hello-bvs1.elf` shows the bytes stored at that address.

For comparison, a static/global `int initialized = 7` typically belongs to `.data`; a static/global `int zeroed` initially has value zero and typically belongs to `.bss`. Static storage lasts for the execution of the program, unlike the local state of a particular call. Neither declaration is needed in the Exercise 2 application merely to fill a diagram.

In the inspected Exercise 2 image, `.data` is empty. The local `value` array belongs to neither section: it lives on the process stack while `worker()` is active. Its `.bss` contains `os_context` and `current_process` from the provided library. A pointer becoming nonzero at runtime does not move its storage to `.data`. BSS occupies memory even though the ELF does not store all its initial zero bytes as file contents; loading/startup must establish the required initial state.

These roles do not imply that our minimal system enforces read/write permissions or separate address spaces. The course linker script and runtime storage choices determine the layout we are studying.

4.8 The process stack is an array in an object

Exercise 2 declares:

```
void main(void)
{
    struct process p;
    /* create, run, inspect state, run again */
    /* ... */
}
```

Here `p` is an automatic object in `main()`'s OS-stack allocation. The embedded array `p.stack` is part of that object. It reserves a subregion that becomes the process's active call stack when `sp` selects an aligned position inside it.

This gives the process distinct stack storage for its execution without creating another ELF section. While the process runs, OS-side calls are suspended. When the OS regains control, it uses other stack space and leaves `p.stack` intact. `main()` remains active throughout, so the process object and its embedded storage remain alive.

If `p` were instead static/global, its containing storage would be static. The role of its stack array would remain process-owned runtime storage. Therefore "process stack" answers what the memory is used for; the declaration and allocation answer where that memory comes from. In this example no dynamic allocation is required.

4.9 Inspect the file without confusing it with live memory

After a successful Exercise 2 build, run these commands in the VM:

```
riscv64-unknown-elf-objdump -h build/hello-bvs1.elf
riscv64-unknown-elf-nm -n build/hello-bvs1.elf
riscv64-unknown-elf-objdump -d \
  --disassemble=context_switch build/hello-bvs1.elf
```

The first command lists sections; the second lists symbols by address; the third decodes the selected function's instructions. In `nm`, `T/t` identifies code symbols and `B/b` identifies BSS-related symbols. A symbol can simply name an address: `_stack_top` is the linker-defined initial OS-stack address, not a C array declaration. These inspection options are documented in the GNU Binutils manuals.

The commands inspect the ELF on disk. They do not display the running target's CPU registers or the live contents of the local array. Debugging information can describe local variables, but that is different from a linker symbol or an observation of runtime memory.

The normal build uses `-O2`. It can inline `permute()`, meaning it incorporates the helper's work into the caller instead of retaining a separate call. Local scalar values such as `tmp` can be kept in registers. The lecture's separate observation build uses `-O0 -g -fno-inline` to retain inspectable function boundaries; `-g` adds debugging information. Exact frames and addresses depend on the build. See GCC's Optimize Options.

4.10 The next requirement: regain control without cooperation

Our current mechanism depends on the process reaching yield or exit. If it runs forever before either operation, our target-side OS continuation cannot run again. Ubuntu can still manage the QEMU process, but that is control at another layer. Terminating QEMU ends the target session rather than implementing target-side process management.

An **interrupt** signals an event. When enabled, it transfers control to a configured OS handler, allowing the OS to regain control. A future **timer interrupt** can make control transfer possible without a voluntary yield. This enables **preemption**: the OS takes control without the process's cooperation. Scheduling then decides whether to continue the interrupted execution or select another runnable one. Interrupt handling and the scheduling decision are related but distinct steps.

This chapter raises that question without requiring an interrupt implementation for CP1. An arbitrary interruption is not an ordinary C call boundary. The interrupted continuation is not supplied by `ra` as it is at our cooperative call boundary; `ra` holds whatever value the interrupted code was using. The interrupted program counter must therefore be preserved explicitly, in dedicated trap state, together with the other registers the interrupted code may be using. The cooperative CP1 register subset is not by itself the full specification of such a mechanism.

4.11 Key ideas

- The Ubuntu VM is our development host; the emulated RV64 machine is the target that executes our program's instructions.
- Editing, saving, building, and starting a new run are separate steps in one observable workflow.
- Function instructions remain in `.text`; stack frames contain runtime data associated with active calls.
- A local writable array is distinct from constant data used to initialize it or print an output label.
- Process stack storage is an array in the process object; its role does not prescribe an ELF section or an address space.
- Preserved stack data and saved register context together support continuing the same execution.
- The program counter is CPU state but not a general-purpose register: the cooperative switch re-establishes it through `ra` and `ret`, whereas an interruption must preserve it explicitly.
- Lifecycle state determines whether another run is permitted, even if old memory contents remain.

Related Exercise and Handouts

Exercise 3 — Tracing Code and Runtime State investigates the workflow and memory roles using the existing Exercise 2 application.

The editor handout and development-environment handout support everyday work in the VM.

References

- GNU Binutils, objdump and nm.
- GCC, Optimize Options.
- RISC-V International, RISC-V ABIs Specification.
- Arpaci-Dusseau and Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*.